

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 3 · Programación Orientada a Objetos

Clases y objetos. Constructores

Encapsulación: modificadores de acceso, getters y setters

Atributos estáticos y métodos estáticos

Sobrecarga de métodos y constructores

La palabra reservada this. Visibilidad y alcance

Estructuras de datos con objetos (ArrayList)

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 2
Valoración	POO: clases, objetos y encapsulación

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 cada pregunta

1. ¿Qué palabra clave se usa para impedir que una clase sea heredada?
- a) static
 - b) final
 - c) abstract
 - d) private

RESPUESTA / SOLUCIÓN

b) final — Una clase marcada como final no puede ser heredada.

-
2. Un atributo declarado como `protected` es accesible desde:
- a) Solo la misma clase
 - b) Cualquier clase del programa
 - c) La misma clase, paquete y subclases
 - d) Solo la clase padre

RESPUESTA / SOLUCIÓN

c) protected — accesible desde la misma clase, el mismo paquete y las subclases (incluso en otros paquetes).

-
3. ¿Cuál es el valor de `contador` tras ejecutar este código?

```
class A {  
    static int contador = 0;  
    A() { contador++; }  
}  
  
public class Main { public static void main(String[] a) {  
    new A(); new A(); new A();  
    A.contador = A.contador + 1;  
    System.out.println(A.contador);  
} }
```

- a) 3
- b) 4
- c) 2
- d) 1

RESPUESTA / SOLUCIÓN

b) 4 — Cada `new A()` incrementa el atributo estático (3), y luego se suma $1 \rightarrow 4$.

-
4. Un campo `private` de una clase puede leerse desde otra clase del mismo paquete sin usar getters.

RESPUESTA / SOLUCIÓN

Falso — `private` solo es accesible desde la propia clase; las demás clases, incluso del mismo paquete, requieren accesorios (getter/setter).

-
5. La sobrecarga de métodos se define como:
- a) Métodos que se llaman igual pero con distinta firma
 - b) Métodos que se heredan
 - c) Métodos con distinto nombre e igual parámetros
 - d) Métodos que devuelven el mismo tipo

RESPUESTA / SOLUCIÓN

a) Sobrecarga = mismo nombre, distinta lista de parámetros (tipo/número). No depende solo del tipo de retorno.

-
6. ¿Cuál es la función de `this` dentro de un constructor?
- a) Referirse al objeto actual
 - b) Llamar al constructor padre
 - c) Devolver el valor actual de la variable
 - d) Crear un nuevo objeto

RESPUESTA / SOLUCIÓN

a) `this` refiere al objeto actual y permite distinguir atributos de parámetros del mismo nombre.

7. ¿Qué imprime este código?

```
class Persona {  
    String nombre;  
    Persona(String nombre) { this.nombre = nombre; }  
}  
  
public class Main {  
    public static void main(String[] a) {  
        Persona p1 = new Persona("Ana");  
        Persona p2 = p1;  
        p2.nombre = "Luis";  
        System.out.println(p1.nombre);  
    }  
}
```

- a) Ana
- b) Luis
- c) null
- d) Error

RESPUESTA / SOLUCIÓN

b) Luis — p2 apunta al mismo objeto que p1 (se copia la referencia), por lo que `p2.nombre = "Luis"` cambia el nombre del objeto compartido, y p1.nombre es "Luis".

8. ¿Qué constructor se invoca con `new Animal()` si la clase Animal define un constructor con parámetros?

- a) El constructor con parámetros por defecto
- b) Error de compilación si no existe constructor sin parámetros
- c) El constructor por defecto implícito
- d) Ninguno

RESPUESTA / SOLUCIÓN

b) Si defines un constructor con parámetros, Java NO genera el constructor sin parámetros automáticamente, por lo que `new Animal()` provoca error de compilación a menos que lo defines.

9. Un método `static` puede acceder directamente a atributos de instancia de la clase.

RESPUESTA / SOLUCIÓN

Falso — Un método estático pertenece a la clase y no puede acceder a atributos de instancia sin un objeto (no existe 'this' en contexto estático).

10. ¿Cuál es la diferencia entre `public`, `private` y `protected`?

- a) Solo afectan a la velocidad del programa
- b) `public` accesible desde cualquier clase; `private` solo desde la propia; `protected` desde paquete y subclases
- c) Son sinónimos en Java
- d) No existen estos modificadores en Java

RESPUESTA / SOLUCIÓN

b) `public` (todos), `private` (solo la clase), `protected` (paquete + subclases).



Parte 2: Análisis y depuración

3 puntos

1. El siguiente código tiene errores de diseño y de sintaxis. Encuéntralos y corrige la clase para que sea correcta y encapsulada. (1,5 pts)

```
public class Cuenta {  
    String titular;  
    double saldo;  
    public Cuenta(String titular, double saldo) {  
        titular = titular;  
        saldo = saldo;  
    }  
    public void depositar(double m) { saldo += m; }  
    public void retirar(double m) { if (saldo >= m) saldo -= m; }  
    public void mostrar() { System.out.println(titular + " " + saldo); }  
}
```

RESPUESTA / SOLUCIÓN

1. Faltan los modificadores `private` en titular y saldo → debe ser `private String titular; private double saldo;`.
2. En el constructor, `titular = titular;` no hace nada (asigna el parámetro a sí mismo). Debe ser `this.titular = titular; this.saldo = saldo;`.
3. Para una buena encapsulación, faltan getters/setters. Se recomienda añadir `getTitular()`, `getSaldo()`, etc., y validar en `retirar`.

-
2. Determina la salida del siguiente programa. (1,5 pts)

```
class Contador {  
    static int total = 0;  
    int valor;  
    Contador() { total++; valor = total; }  
    void mostrar() { System.out.println("v=" + valor + " total=" + total); }  
}  
  
public class Main {  
    public static void main(String[] a) {  
        Contador c1 = new Contador();  
        Contador c2 = new Contador();  
        Contador c3 = new Contador();  
        c1.mostrar(); c2.mostrar(); c3.mostrar();  
    }  
}
```

RESPUESTA / SOLUCIÓN

total es estático (compartido): 1, luego 2, luego 3.
valor es de instancia: c1.valor=1, c2.valor=2, c3.valor=3.
Salida: `v=1 total=3`, `v=2 total=3`, `v=3 total=3`.

Parte 3: Ejercicios de programación

7 puntos

1. Diseña la clase `Rectángulo` con atributos privados `ancho` y `alto` (double). Incluye: constructor, getters/setters, métodos `area()`, `perimetro()`, y `toString()`. Además, sobrecarga el constructor para permitir crear un cuadrado con un solo parámetro. Escribe un programa de prueba. (2,5 pts)

RESPUESTA / SOLUCIÓN

```
public class Rectangulo {
    private double ancho, alto;

    public Rectangulo(double ancho, double alto) { this.ancho=ancho; this.alto=alto; }
    public Rectangulo(double lado) { this(lado, lado); } // cuadrado
    public double getAncho() { return ancho; }
    public void setAncho(double ancho) { this.ancho = ancho; }
    public double getAlto() { return alto; }
    public void setAlto(double alto) { this.alto = alto; }
    public double area() { return ancho * alto; }
    public double perimetro() { return 2*(ancho+alto); }
    public String toString() { return ancho + "x" + alto; }
}

// Main:
public class Main { public static void main(String[] a) {
    Rectangulo r = new Rectangulo(4, 6);
    Rectangulo c = new Rectangulo(5);
    System.out.println(r + " área " + r.area());
    System.out.println(c + " perímetro " + c.perimetro());
}}
```

2. Crea la clase `Estudiante` con atributos `nombre`, `edad` y una lista de notas (`ArrayList<Double>`). Añade métodos para añadir nota, calcular la media, determinar si está aprobado (media ≥ 5) y mostrar sus datos. (2,5 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.ArrayList;

public class Estudiante {
    private String nombre;
    private int edad;
    private ArrayList<Double> notas = new ArrayList<>();
    public Estudiante(String nombre, int edad) { this.nombre=nombre; this.edad=edad; }
    public void addNota(double n) { notas.add(n); }
    public double media() {
        double s=0; for (double n: notas) s+=n; return s/notas.size(); }
    public boolean aprobado() { return media() >= 5; }

    public String toString() { return nombre + " (" + edad + ") media=" + media(); }
}
```

-
3. Crea una clase `CuentaBancaria` con titular, saldo y número de cuenta. Los métodos retirar y depositar deben actualizar el saldo. Un atributo estático `totalCuentas` debe contar cuántas cuentas se han creado. Escribe un programa que cree varias cuentas, realice depósitos y retiradas, y muestre el total de cuentas. (2 ptos)

RESPUESTA / SOLUCIÓN

```
public class CuentaBancaria {
    private String titular;
    private double saldo;
    private static int totalCuentas = 0;
    public CuentaBancaria(String titular, double saldo) { this.titular=titular; this.saldo=saldo;
totalCuentas++; }
    public void depositar(double m) { saldo += m; }
    public boolean retirar(double m) {
        if (saldo >= m) { saldo -= m; return true; }
        return false;
    }

    public static int getTotalCuentas() { return totalCuentas; }
    public String toString() { return titular + ": " + saldo; }
}
```


Parte 4: Pregunta teórica

1 punto

1. Explica el concepto de encapsulación. ¿Por qué es importante declarar los atributos como ``private`` y proporcionar getters/setters? ¿Qué beneficios aporta al mantenimiento del código?

RESPUESTA / SOLUCIÓN

La encapsulación es el principio que oculta el estado interno de un objeto y solo expone una interfaz pública controlada.

Declarar atributos `private` evita que la clase se modifique desde fuera sin control, permitiendo validar los valores (p. ej., impedir saldos negativos) y cambiando la implementación interna sin romper a los clientes de la clase.

Beneficios: seguridad, control de acceso, mantenibilidad, y facilita el testing y la evolución del código.



Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)